HospitalOS

Hospital Operating System for Africa

HOSPITALOS DOCUMENTATION

Platform Admin Guide

Internal operator reference

VERSION

3.1 · July 2026

PREPARED BY

John Ipadeola, Production Team Lead

AgoroX Africa

1. Who this guide is for

This guide is for the platform admin — the AgoroX Africa account that manages the HospitalOS platform across every tenant hospital, separately from any single hospital's own staff.

Role	Scope	Account
Platform admin	The whole platform — all tenants, settlement, usage	support@agorox.africa

Your account. *support@agorox.africa is the platform admin account. Keep this credential strictly confidential — it carries visibility into every tenant's usage and settlement data, and is the only account the platform-only routes described in this guide will accept.*

2. Signing in as platform admin

1 Open the HospitalOS sign-in screen.

2 Enter support@agorox.africa and the platform admin password — checked through the same server-side, securely-hashed verification every account uses.

3 A Hospital / Platform toggle appears in the topbar, visible only to this account.

4 Click it to switch into Platform mode.

Access control, not just visibility. *The toggle isn't merely hidden by styling from other accounts — every platform-only backend route explicitly rejects any request from an account that isn't flagged as platform admin, checked on the server on every single call. A tenant Super Admin cannot reach these screens by guessing a URL.*

Signing out ends the session on the server. *Signing out doesn't just clear this browser — it revokes the session record on the server the moment you sign out, so the token is dead immediately and cannot be reused even if it had time left before its automatic expiry. On a shared machine this genuinely closes platform access rather than leaving a reusable session behind.*

3. Onboarding a new tenant

Every activation starts with a code, generated by the platform admin and issued to the hospital — never self-service, and never guessable.

1 Generate an activation code from Platform → Tenants, setting the hospital's name and plan tier at creation.

2 The code is genuinely single-use: redeeming it flips its status from unredeemed to redeemed through an atomic, race-condition-safe update — two people attempting to redeem the same code at the same moment cannot both succeed, one gets a clear “already redeemed” response.

3 A code can also expire before it's ever used, checked against a real expiry timestamp — an expired, unredeemed code is reported distinctly from an already-redeemed one, so support can tell the two situations apart immediately.

4 Redemption creates the hospital's own tenant record and its first Super Admin account in one step — that account then runs its own onboarding wizard (branding, initial pricing, staff accounts) entirely within its own tenant, with no further platform-admin involvement needed.

Demo tenants use the same mechanism, deliberately time-boxed. *A self-service demo tenant is created through a separate, public endpoint with a fixed 7-day expiry — useful for a live sales demonstration without platform-admin involvement, but not intended as a long-lived account. If a* *demo needs to persist past 7 days for an extended evaluation, that expiry is a real database field that can be extended manually; it is not a hidden or silently-renewing timer.*

4. The Platform navigation — what each screen does

4.1 Overview

Real tenant count, seats, active users, and platform health at a glance, read live from the tenants and accounts tables — not a separate, disconnected summary.

4.2 Tenants

Every real tenant deployment with plan, seats, active users, and status, queried directly from the tenants table — not a fixed illustrative list. As of this writing the platform has one live tenant, Ibadan Teaching Hospital, on the Enterprise plan; this table grows as new hospitals activate.

MRR is honestly blank, not estimated. *Monthly recurring revenue isn't shown as a number here because no real recurring-billing aggregation exists yet to compute it from — tenants are on a mix of flat and commission billing, and inventing a figure would be worse than leaving it open. See Section 4.3 for what actually is computed.*

4.3 Settlement centre

Commission is genuinely per-tenant now, not one platform-wide rate — each hospital's own contract (flat annual, or a percentage of collections) determines what it owes, calculated the moment each of its payments is recorded into a ledger kept completely separate from that hospital's own revenue records. Cycles run twice a month: the 1st–15th and the 16th–end of each month. Status moves Pending → Processing → Settled.

Verified before shipping. *The cycle boundary math was tested directly against real edge cases — a year boundary (December into January), a mid-cycle date, and February's 28-day month all landed on the correct dates — not just assumed correct.*

4.3.1 Commercial plans

A tenant can also be on a richer plan than the simple flat/percentage split: a minimum monthly fee floor, a maximum cap, a temporary promotional rate, or progressive tiered commission (the same logic as tax brackets — each portion of gross is charged at its own tier's rate, not the whole amount retroactively). This is applied as a clearly labelled adjustment at the moment a cycle closes, on top of the simple per-payment calculation, never a silent rewrite of it — both the simple figure and the adjusted figure are kept on record.

4.3.2 Platform invoices

A real invoice is generated automatically for every tenant that owed a non-zero fee when its cycle closes — no manual step. "Mark paid" is a manual confirmation once payment is actually received through some other channel, and it is idempotent — a double-click or repeated confirmation cannot mark the same invoice paid twice. Genuinely automated collection needs a payment method on file, which isn't built.

4.3.3 Revenue mix by module

Real now, but only for invoice-linked payments: an invoice's own line items carry which module each charge came from, so a payment taken against one can be prorated back across those sources. A direct payment with no invoice can't be traced this way and is shown honestly as unattributed rather than guessed at — in practice, expect this to be the majority of revenue until tenants generate invoices more often, since nothing in the cashier flow requires one.

4.4 Usage analytics

Per-tenant seats, active users, and real activity counts — patients registered, lab orders, imaging studies, prescriptions dispensed — queried live from each tenant's own data, not estimated.

Churn signal. *A tenant using less than half its purchased seats is flagged explicitly. This is not decorative — it is the earliest, cheapest point to have a plan-sizing conversation, before a tenant's usage drops further.*

4.5 Platform health

Service	Role
Web app (Cloudflare Pages)	Serves the built frontend
API Worker	Backend request handling
D1 database	Persistent tenant data — real, live, and the backing store for every module in the Tenant Guide
R2 object storage	Real file storage for Documents & templates uploads
HL7 instrument gateway	Device management and monitoring are real; live analyzer/device network listening is still simulated — see Section 6.3
Email (Resend)	Transactional email delivery

4.6 Feature flags

Unlocks or withholds specific modules per plan tier, backed by a real table — toggling a flag here persists and takes effect immediately, useful for phased rollouts without a separate deployment.

5. Downloading this guide

This guide is available at any time from the Platform Overview screen — click “Platform Admin Guide (PDF)” in the top-right actions bar. It is not linked anywhere a tenant user could reach: switching into Platform mode itself is refused outright for any account without platform admin rights, so this is not merely hidden by styling.

6. Architecture for support staff

You do not need to read code to support tenants, but this shape explains why certain things behave the way they do when a tenant reports an issue.

6.1 The platform runs on a real backend

Every module — across every one of the thirteen tenant-facing groups described in the Tenant Guide — persists in Cloudflare D1. A tenant's data survives a reload, a different device, a different staff member's session. This was the outcome of a full, deliberate migration effort; it is not a future plan.

6.2 “Engines” vs. modules

Help and Pricing are structured as standalone engines rather than ordinary feature modules: an engine owns its own state, is imported by other modules, and never imports a module itself. This is why a price override set in Administration → Pricing takes effect immediately across every billing calculation and every screen showing that price, from one single source of truth.

6.3 The instruments gateway simulates live listening

Device management, registry, and monitoring are real and persist. What remains simulated is the network listener itself — an MLLP socket, a DICOM SCP, a print daemon — because that's server/network infrastructure genuinely beyond what a browser-based feature stands up. Every action calls the exact downstream function a live listener would call.

7. RBAC and the audit chain, explained for support

Every role has an AREA list and a GRANTS list of area:action permission strings, enforced on the server for every request. When a tenant reports "my nurse can't discharge a patient," check the role matrix under Administration → Users & roles before treating it as a bug — it is very likely by design.

The audit log is append-only and hash-chained: each entry links to the hash of the one before it, stored in the database, not the browser. If a tenant asks whether their audit log can be trusted, the honest answer is that it is tamper-evident — the standard, honest claim for this kind of log — not a claim that database-level tampering is physically impossible for someone with that access, which no audit design claims.

A real, separate finding worth knowing. *Earlier in this platform's build, the Security & Audit screen was found to be reading from a disconnected client-side log instead of the real audit_log table — discovered and fixed. If a tenant ever reports an audit screen that looks thin or disconnected from real activity, that class of bug is exactly what to check for first.*

7.1 A worked example, for training a new support hire

A tenant's Nurse account reports: "I opened a patient's record but there's no Discharge button." Walk it through: Nurse's grants list does not include patient-care:discharge — confirmed in the role matrix, not assumed. This is Doctor's grant specifically, a deliberate separation of duties (the same principle as Cashier being unable to approve its own claim — see the Support runbook, Section 9). The correct response is to explain the boundary, not to file it as a bug — and if the tenant genuinely wants that Nurse to discharge patients, the fix is a role change on their end, under their own Administration → Users & roles, not a platform-side override.

8. Current state, stated honestly

Read this before promising a prospective tenant anything about what's built. Everything not listed here — including every module in the Tenant Guide, real authentication, real file storage, and the real audit trail — is genuinely live.

- Lab/imaging/radiotherapy devices — simulated message flow. Device registry and monitoring are real; live MLLP/DICOM/print-daemon network listening needs real network infrastructure a browser can't provide.
- No real SMS/WhatsApp/email carrier connected. The message queue, templates, and channel selection are real; delivery stays honestly at "queued."
- Revenue mix by module in Settlement is only real for invoice-linked payments — a direct payment can't be traced to a specific module and shows as unattributed.

 Platform invoice generation is automated at settlement close; collection is still a manual confirmation — automatically charging a tenant needs a payment method on file, not built yet.

 No self-service password reset for any account, tenant or platform — a lost credential needs manual reset. When an account is deactivated, or its access is withdrawn, every session it currently holds is revoked at once rather than lingering until expiry.

 Documents & templates needs a real Cloudflare R2 bucket per deployment — already true for the current live deployment, but worth confirming during any new environment setup.

Online payments are live. *Card / online checkout runs through Paystack on the current deployment: the provider abstraction, the checkout initialisation, and the webhook are all wired, with the webhook verifying the provider's signature and independently re-confirming each transaction against the provider's own API before any payment is recorded. Adding or swapping a provider is a change in one place, not a rewrite of the payment flow.*

None of this is a defect. *Each gap above is a deliberately scoped, named limitation — not architecture that needs to be redone. Everything else described in this guide and the Tenant Guide is real, tested, and live.*

9. Support runbook — exact messages, real causes

What a tenant is actually seeing when they report a problem, drawn from real server responses rather than guesswork — check this before escalating.

Tenant reports…	Almost certainly means…
"I can't take a cash payment"	No open cash session — Cash specifically requires one, every other method doesn't. Not a bug.
"A payment got rejected for no reason"	Server-side balance validation caught an amount exceeding what's genuinely owed against current prices — ask what the exact error text said before assuming a defect.
"My staff member can't see [screen]"	A role/area boundary, almost always — check their role's area list under their own Administration → Users & roles before treating as a bug.
"Settlement's fee doesn't match what I expected"	Check whether the tenant has a commercial plan configured (Section 4.3.1) — a floor, cap, or tiered rate changes the simple calculation, and the adjustment is shown separately, not blended in silently.
"A refund/chargeback option is missing"	Both require the elevated finance:refund grant, deliberately separate from routine payment-taking — confirm the account's grants, not a bug report.
"A staff member is still logged in after we deactivated them"	Should not happen — deactivation now revokes their live sessions immediately. If it recurs, treat as Critical and escalate.

10. Escalation & support

For issues that are genuinely product bugs rather than expected RBAC behaviour or a known limitation (Section 8), escalate with: the tenant name, the account role in use, the exact screen and action, and whether it reproduces on a fresh sign-in.

Severity	What it looks like	Response
Critical	Money, patient safety, or data integrity affected — a payment recorded wrong, an allergy check not blocking, an audit entry that doesn't verify	Immediate — do not wait for a scheduled check-in
High	A whole workflow blocked for a tenant — nobody can sign in, a whole module returns errors	Same business day
Normal	A single account or edge case affected, a workaround exists	Standard queue
Not a bug	Expected RBAC boundary, or a named item from Section 8	Explain the design; no escalation needed

When in doubt, treat it as Critical and downgrade later. *The cost of escalating a normal issue too urgently is small. The cost of sitting on a genuine financial or safety issue because it wasn't obviously severe is not — this is the same fail-safe philosophy the platform's own allergy and balance checks are built around.*

11. Security reminders

— Never share the platform admin credential outside AgoroX Africa.

— Any change made while inside a tenant's data is a real change to that hospital's real records — be deliberate.

— Signing out revokes your session on the server; on a shared machine, sign out rather than just closing the tab.

— This guide is distributed only through the Platform view, visible solely to the platform admin account.

Questions about this guide? *support@agorox.africa*